

Instrument Approach Procedure Chart Georeferencing

[ACM Charting Group 26-01-411](#)

2026-08-20

Mark Mentovai
mark@mentovai.com

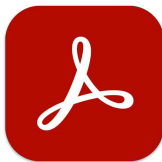
Geospatial PDF

- First defined in a [2008 extension to PDF 1.7](#) (extension level 3, §8.8.1).
- Further standardized by **PDF 2.0** ([ISO 32000-2:2020](#)) §12.10, although PDF 2.0 is not widely adopted.
- Geospatial extension maps commonly used georeferencing concepts onto the PDF document structure.
 - **PROJCS**: projected coordinate system (such as Lambert Conformal Conic) and its parameters.
 - Embeds a **GEOGCS**: geographic coordinate system (such as NAD83/GRS80).
 - **GPTS**: georeferenced geographic (latitude–longitude) coordinates.
 - **LPTS**: corresponding page-centric Cartesian (x, y) coordinates.
 - **Bounds**: the neatline, or area within a page in which georeferencing is valid.
- Comparison to [GeoTIFF](#).
 - First defined in 1995 ([GeoTIFF 1.0](#)).
 - Geospatial PDF PROJCS/GEOGCS correspond to GeoTIFF **GeoKey Directory** (GeoKeyDirectoryTag).
 - Geospatial PDF GPTS/LPTS correspond to GeoTIFF **tiepoints** (ModelTiepointTag).
- **FAA-produced instrument approach charts in d-TPP have been georeferenced as Geospatial PDFs** starting in [2016](#), completed by [2017](#).
 - *Exception*: charts produced by **DoD/NGA** and distributed in TPP by FAA. These are bitmap (raster) images embedded in their PDFs, and are not georeferenced. These are all at military airfields or are high-altitude (military-only) procedures at civil/joint-use fields.
 - *Exception*: **charted visual flight procedures** (CVFP), like “Parkway Visual” and “River Visual”, are not georeferenced.
 - *Exception*: A small number of **continuation pages**.
 - *Exception*: KATL, KDTW, and KORD “**PRM AAUP**” notices, classified as IAPs because there’s no way to identify IAP AAUPs.
 - In cycle 2608, 10,010 out of 11,170 IAP PDFs (**90%**) are Geospatial PDFs.

Demonstration: Open Geospatial PDF in Acrobat



→ open in →



00610IL22L.PDF

Adobe Acrobat

Steps

- Open a Geospatial PDF in Adobe Acrobat (Reader or Pro).
- All tools→(View more)→Measure objects.
- Select "Measuring tool" or "Geospatial location tool".

Drawbacks

- No way to export data.
- **Not a real technique for programmatic data access.**
- This is just a demonstration showing that georeferencing data is in the PDF.

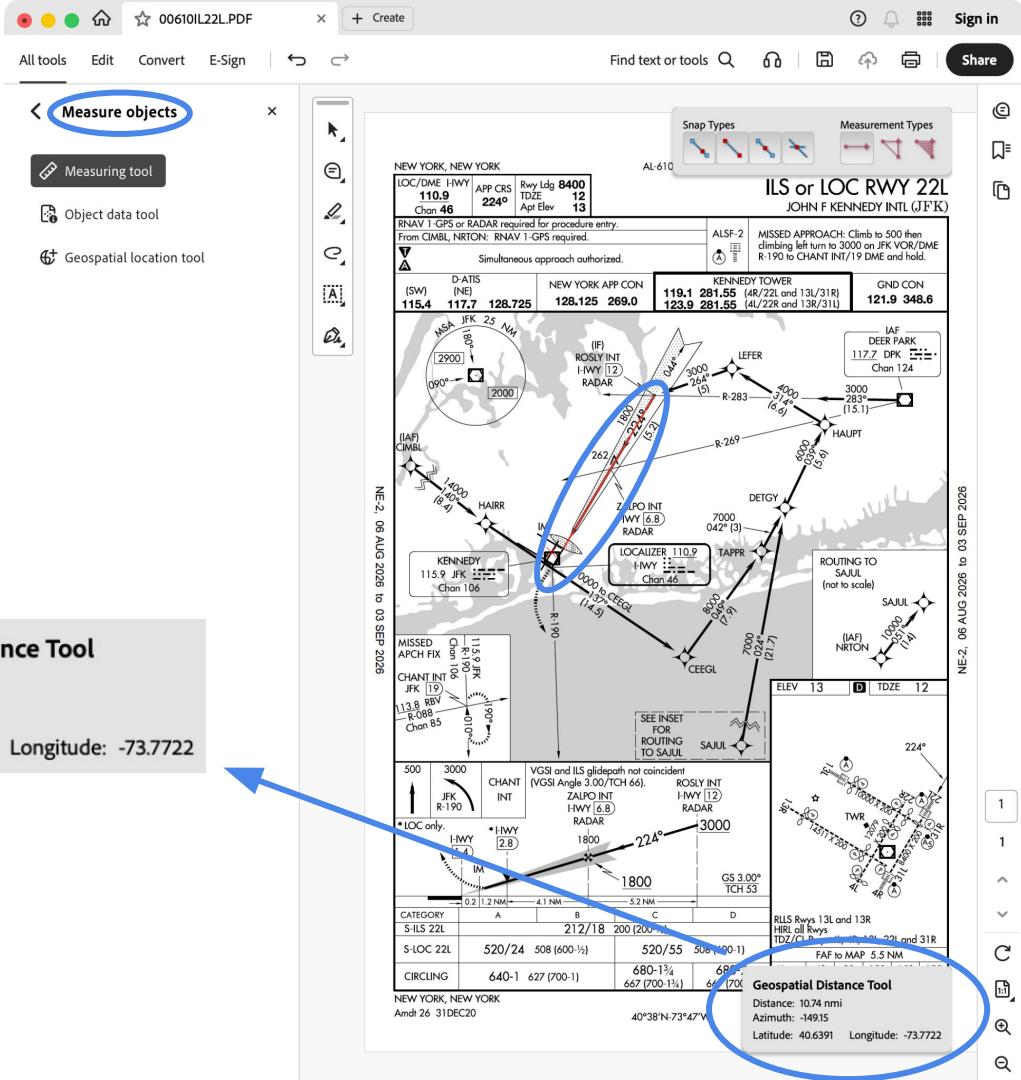
Geospatial Distance Tool

Distance: 10.74 nmi

Azimuth: -149.15

Latitude: 40.6391

Longitude: -73.7722



GDAL (Geospatial Data Abstraction Library)

- Open-source library and tools that operate on geospatial data formats...
 - ...including Geospatial PDF!
 - *and* GeoTIFF and many others, too.
- gdal.org
- Maintained by the Open Source Geospatial Foundation (OSGeo Project, osgeo.org).
- Command-line tools like `gdal info`.
- Library interfaces in many languages.
- Available for all major operating systems.
- MIT-licensed source code available at <https://github.com/OSGeo/gdal>.



Technique: Convert Geospatial PDF to GeoTIFF

Benefits

- GeoTIFF is a **mature container for georeferenced images**.
- Probably most the **broadly supported** format for this purpose.
- Existing tools and pipelines are **likely to interoperate** well with GeoTIFF.

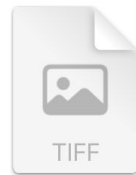
Drawbacks

- Rasterization **loses valuable vector data** (text, clean rescaling, document structure) present in PDF files.
- **File sizes** generally exceed those of PDF, particularly at higher resolutions.
- Guidance:
 - *Downstream, data suppliers:* If you were going to rasterize before distribution anyway, these concerns won't apply.
 - *Upstream, FAA:*
 - I don't recommend pursuing this in place of the existing PDFs, because it constrains downstream uses.
 - However, there's some precedent for publishing a single product in multiple formats. d-VC (visual charts: sectionals, etc.) and IFR charts (enroute, etc.) are available in *both* georeferenced GeoTIFF and (non-georeferenced) PDF.



00610IL22L.PDF

→ gdal convert →

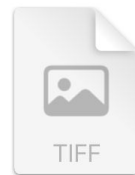


00610IL22L.tiff

Technique: Convert Geospatial PDF to GeoTIFF



→ gdal convert →



00610IL22L.PDF

00610IL22L.tiff

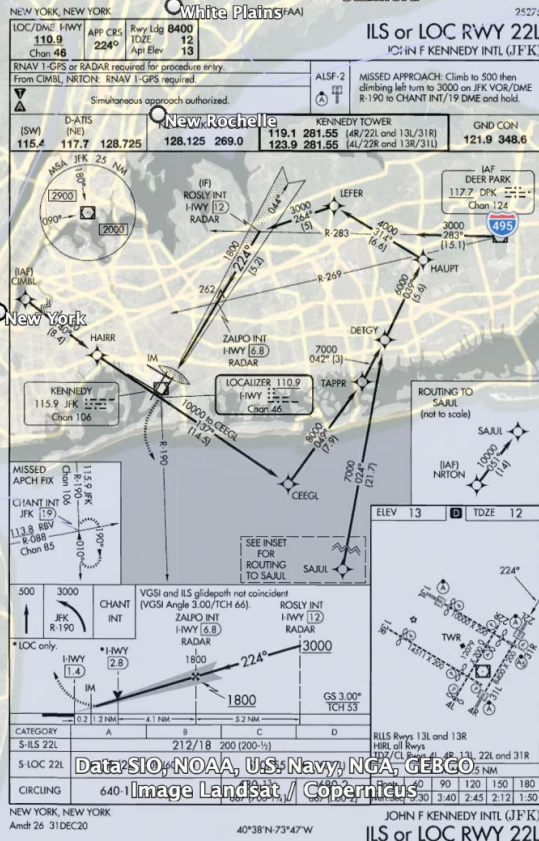
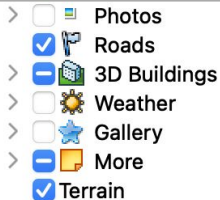
```
% gdal convert \  
  --config GDAL_PDF_DPI=300 \  
  --creation-option COMPRESS=DEFLATE \  
  --creation-option ZLEVEL=9 \  
  00610IL22L.PDF \  
  00610IL22L.tiff  
0...10...20...30...40...50...60...70...80...90...100 - done.
```



```
gdal convert
```



open in

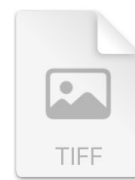


eye alt 76.26 mi

Google Earth



→ gdal convert →



00610IL22L.**PDF**

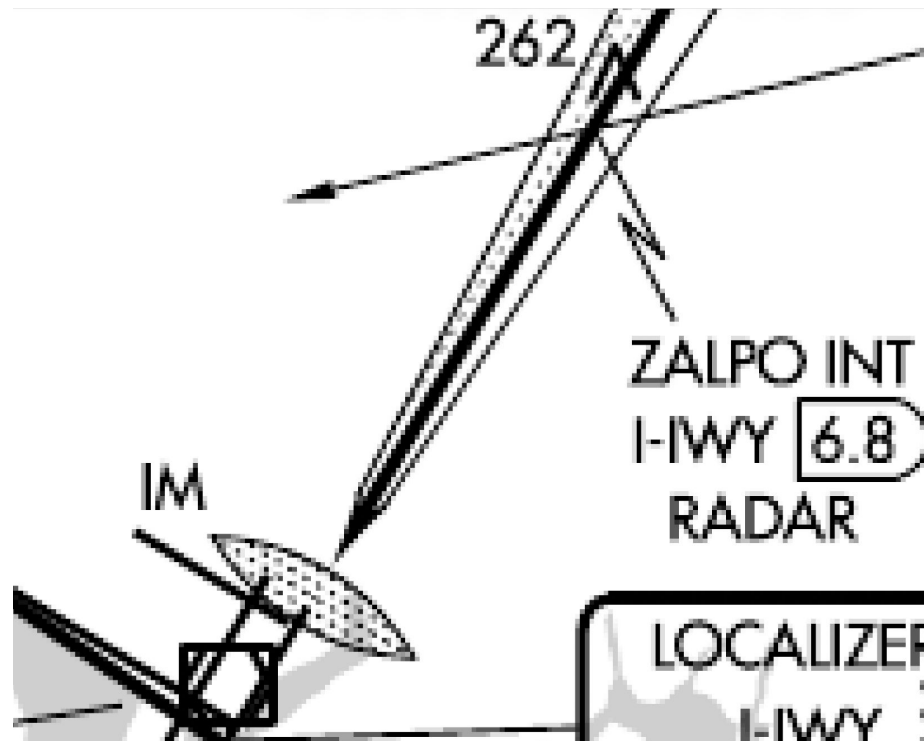
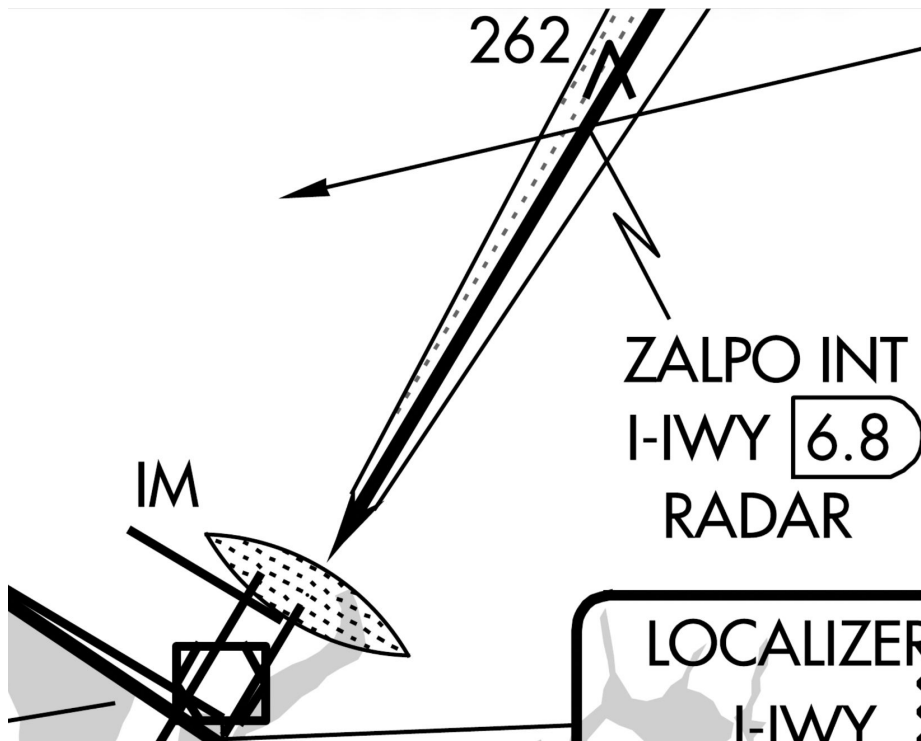
310kB

sharp when zoomed in

00610IL22L.**tiff**

2,930kB (uncompressed) @ 150dpi

pixelated when zoomed in



Technique: Extract Georeferencing Data from Geospatial PDF

- ...and put it *somewhere* in *some format*.
 - Figure that out later.
- This can be done upstream (FAA) and published, or it can be done downstream (data suppliers) for in-house use, using an identical technique.
- Parameters to extract and convey:
 - **Geographic coordinate system**, including datum and ellipsoid. *Always NAD83/GRS80 in FAA IAPs.*
 - Projected coordinate system. *Always Lambert Conformal Conic in FAA IAPs.* Requires parameters:
 - Two **standard parallels of latitude**.
 - **Origin coordinates**: latitude, and central meridian of longitude.
 - **Offsets**: false easting and northing. *Almost always 0 in practice; always 0 in FAA IAPs.*
 - **Control points**/tiepoints, relating geographic coordinates (latitude–longitude pairs) to page-centric Cartesian (x, y) positions on the chart, such as page corners.
- One-off use: run `gdal info`.
- Production pipeline:
 - **Script and scrape** `gdal info` (or `gdal info --format=json`).
 - **Use GDAL as a library**.
 - Other options are possible, addressed later.

Extract Georeferencing Data with gdal info

```
% gdal info --config GDAL_PDF_DPI=300 00610IL22L.PDF
```

```
Driver: PDF/Geospatial PDF
```

```
Files: 00610IL22L.PDF
```

```
Size is 1614, 2475
```

```
Coordinate Reference System WKT:
```

```
PROJCRS["",
```

```
BASEGEOGCRS["NAD83",
```

```
DATUM["North American Datum 1983",
```

```
ELLIPSOID["GRS 1980",6378137,298.2572221,...],...],
```

```
PRIMEM["Greenwich",0,...]],
```

```
CONVERSION["unnamed",
```

```
METHOD["Lambert Conic Conformal (2SP)",...],
```

```
PARAMETER["Latitude of false origin",40.7253055555556,...],
```

```
PARAMETER["Longitude of false origin",-73.6931666666667,...],
```

```
PARAMETER["Latitude of 1st standard parallel",44.6666666666667,...],
```

```
PARAMETER["Latitude of 2nd standard parallel",39.3333333333333,...],
```

```
PARAMETER["Easting at false origin",0,...],
```

```
PARAMETER["Northing at false origin",0,...]],
```

```
CS[Cartesian,2,
```

```
AXIS["(E)",east,ORDER[1],...],
```

```
AXIS["(N)",north,ORDER[2],...]]
```

```
[...]
```

```
Corner Coordinates:
```

```
Upper Left (-1081155.295, 1554888.755) ( 74d 1'12.98"W, 41d 4'50.83"N)
```

```
Lower Left (-1081155.295,-2570102.995) ( 74d 0'56.06"W, 40d 8'11.55"N)
```

```
Upper Right ( 1608839.325, 1554888.755) ( 73d12'23.08"W, 41d 4'48.76"N)
```

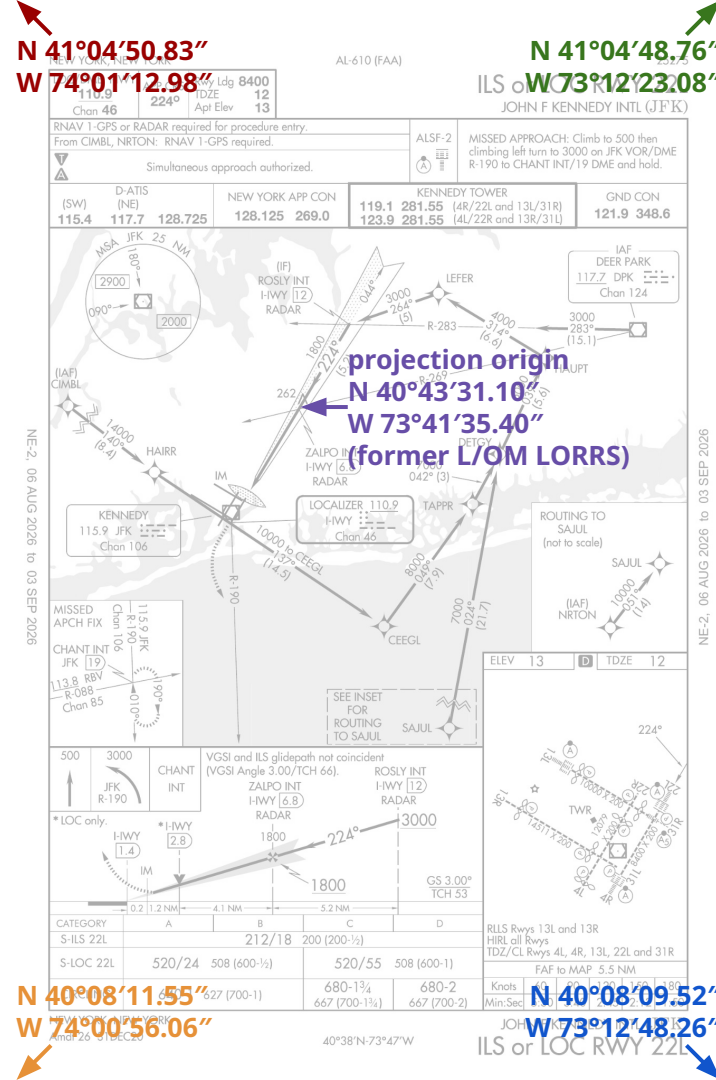
```
Lower Right ( 1608839.325,-2570102.995) ( 73d12'48.26"W, 40d 8' 9.52"N)
```

```
Center ( 263842.015, -507607.120) ( 73d36'50.11"W, 40d36'32.68"N)
```

```
[...]
```



00610IL22L.PDF



Output Format: Projection and Parameters

- **WKT** (well-known text).
 - This is the format stored in the PDF (in particular, WKT1-ESRI).
 - **Flexible:** allows arbitrary projections and their parameters.
 - All (or nearly all) **GIS software understands this format.**
 - Generic parsing requires a schema.
 - There are some ambiguities in the way data is conveyed, particularly in the widely used WKT1.
- **PROJ.**
 - This format is used by the PROJ library.
 - **Flexible:** allows arbitrary projections and their parameters.
 - Some GIS software understands this format.
 - Parsing is straightforward, and encoding is largely unambiguous.
- ~~GeoJSON~~ *(infeasible)*.
 - Raised as a possibility during ACM CG 26-01.
 - Originally (2008), [GeoJSON had a way to express projections in crs objects](#), but **this feature was removed from GeoJSON (RFC 7946, 2016-08)**.
 - GeoJSON crs only provided a location to convey the projection, but for the intended purpose here (a projection with parameters), **the definition would need to appear in WKT or PROJ** anyway.
- **Hard-coded Lambert Conformal Conic**, conveying only its parameters.
 - Inflexible: **only supports this single projection method.**
 - Parameters are broken out, so they're **easy to access.**
 - **No software will understand this out of the box.** The parameters will always need to be marshalled into something else for use.

Output Format: Projection and Parameters, Examples

Each of these are equivalent.

- **WKT** (well-known text) (*WKT1-ESRI, as used in d-TPP PDFs, shown here*).
 - PROJCS["",GEOGCS["GCS_North_American_1983",DATUM["D_North_American_1983",SPHEROID["GRS_1980",6378137.0,298.2572221]],PRIMEM["Greenwich",0.0],UNIT["Degree",0.0174532925199433]],PROJECTION["Lambert_Conformal_Conic"],PARAMETER["False_Easting",0.0],PARAMETER["False_Northing",0.0],PARAMETER["Central_Meridian",-73.6931666666667],PARAMETER["Standard_Parallel_1",44.6666666666667],PARAMETER["Standard_Parallel_2",39.3333333333333],PARAMETER["Latitude_Of_Origin",40.7253055555556],UNIT["Inch",0.0254000508001]]
- **PROJ** (*PROJ4 shown here*).
 - +proj=lcc +lat_0=40.7253055555556 +lon_0=-73.6931666666667
+lat_1=44.6666666666667 +lat_2=39.3333333333333 +x_0=0 +y_0=0 +datum=NAD83
+units=us-in +no_defs=True
- **Hard-coded Lambert Conformal Conic**, conveying only its parameters.
 - Projection: Lambert Conformal Conic
 - Datum: NAD83
 - Standard Parallels: 44.6666666666667, 39.3333333333333
 - Origin (Latitude, Longitude): 40.7253055555556, -73.6931666666667

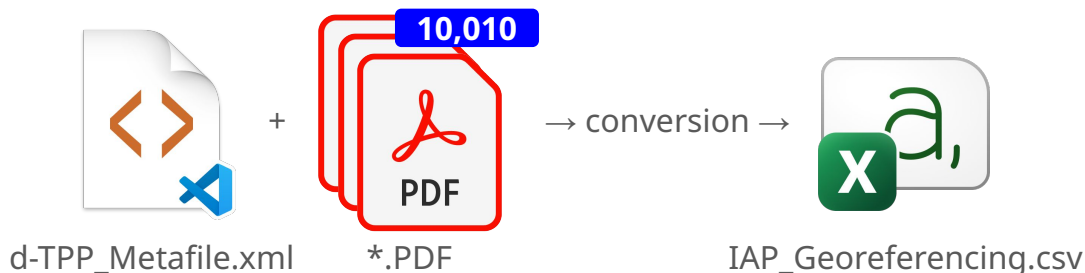
Output Format: CSV

Benefits

- This is exactly what the recommendation document requests.
- It's **easy to parse**.

Drawbacks

- **There's no standard** for georeferencing data in CSVs, so whatever is produced would be “invented”.
- Although easy to parse, all users would require **custom tooling** to harness and use the data.
- **Inflexible:** CSV does not lend itself to the rich set of possible transforms, so the designed solution would necessarily be tied to the details of today's georeferenced charts.
- In the example that follows, unchanging parameters (projection method, datum/ellipsoid, false easting/northing) are omitted. These **must be documented**.
- *Upstream, FAA:* It's another file to produce and host somewhere for distribution.
- It's another file for users to have to obtain and match up with the data.



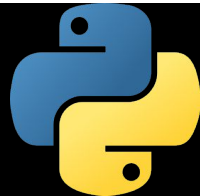
Output Format: CSV, converter pseudocode

- **Write the CSV header**, giving column names.
- For each <record> in d-TPP_Metafile.xml with <chart_code> = "IAP" and <useraction> ≠ "D" (deleted):
 - **Open the chart PDF**, using the filename from <pdf_name>.
 - If the PDF is a Geospatial PDF:
 - Assert that the coordinate reference system uses the Lambert Conformal Conic projection, datum NAD83, and no false easting/northing.
 - Note **projection parameters**: two standard parallels and origin (latitude, longitude).
 - Determine the **projected Cartesian (x, y) coordinates of the page corners** as distances from the projection's origin.
 - Apply the projection's reverse transformation to convert those projected Cartesian (x, y) corner coordinates to **geographic (latitude-longitude) coordinates**.
 - **Write an output record** for the chart, containing its projection parameters and corner geographic coordinates.

Don't Panic!

- Some code examples follow.
 - Don't touch that dial! **You don't have to be a software engineer** to follow along.
 - The samples are functionally equivalent to the pseudocode, but directly usable in practice.
 - I'm showing code to make clear **how easy it is to access the data...**
 - **...and how little code it takes.** Each example fits on a single slide.
- The examples use rasterio.
 - An open-source Python library wrapping GDAL.
 - Installable from PyPI [rasterio package](#) (pip install rasterio).
 - Documentation: [rasterio.readthedocs.io](#).
 - BSD-licensed source code available at <https://github.com/rasterio/rasterio>.
- The source code for these examples is available at https://github.com/markmentovai/faa_tpp_iap_georef/ under an Apache 2.0 license.

Output Format: CSV, converter Python code



```
def faa_tpp_iap_georef_csv(tpp_dir: os.PathLike[str] | str, csv_file: typing.TextIO) -> None:
    csv_writer = csv.writer(csv_file)

    # Write the CSV header.
    csv_writer.writerow(('filename', 'sp_lat_1', 'sp_lat_2', 'origin_lat', 'origin_lon', 'tl_lat', 'tl_lon', 'bl_lat', 'bl_lon',
                        'tr_lat', 'tr_lon', 'br_lat', 'br_lon'))

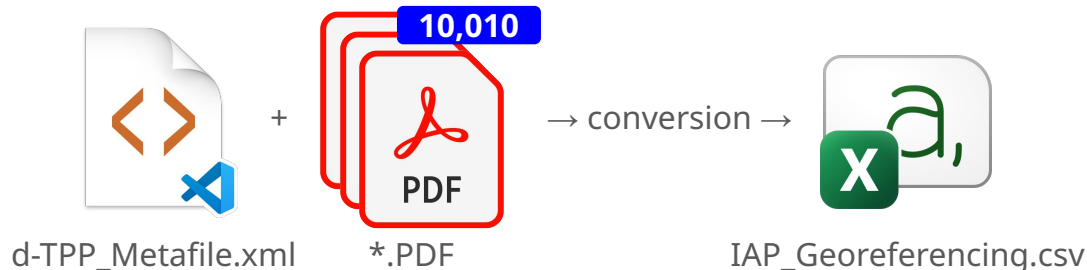
    # Scan d-TPP_Metatile.xml for charts. Only look at IAPs, and don't look at anything that's been deleted.
    metatile = xml.etree.ElementTree.parse(os.path.join(tpp_dir, 'd-TPP_Metatile.xml'))
    for chart_el in metatile.iterfind('./state_code/city_name/airport_name/record/chart_code[.="IAP"]/../../useraction[!="D"]/..'):
        pdf_name_el, = chart_el.iterfind('./pdf_name')
        pdf_name = pdf_name_el.text
        assert isinstance(pdf_name, str)
        with warnings.catch_warnings(), rasterio.Env(GDAL_PDF_DPI=300):
            warnings.simplefilter('error', rasterio.errors.NotGeoreferencedWarning) # Allow non-georeferenced plates to be skipped.
            try:
                with rasterio.open(os.path.join(tpp_dir, pdf_name)) as rio:
                    crs_dict = rio.crs.to_dict() # In production, assert to ensure Lambert Conformal Conic and NAD83/WGS84.

                    # Corner coordinates. "xy" are Cartesian, distances from the origin.
                    xy_tl = rio.xy(0, 0, offset='ul')
                    xy_bl = rio.xy(rio.height - 1, 0, offset='ll')
                    xy_tr = rio.xy(0, rio.width - 1, offset='ur')
                    xy_br = rio.xy(rio.height - 1, rio.width - 1, offset='lr')

                    # Corner coordinates. "ll" are geographic (latitude/longitude).
                    ll_wgs84 = rasterio.warp.transform(rio.crs, crs_dict['datum'], # 'NAD83', or use 'WGS84' or 'EPSG:4326'
                                                    (xy_tl[0], xy_bl[0], xy_tr[0], xy_br[0]), (xy_tl[1], xy_bl[1], xy_tr[1], xy_br[1]))
                    ll_tl, ll_bl, ll_tr, ll_br = ((ll_wgs84[1][i], ll_wgs84[0][i]) for i in range(4))

                    # Write this chart's information to the CSV output. 7 decimal places is .00036" resolution, better than dd°mm'ss.sss", ≤1.1cm.
                    csv_writer.writerow((pdf_name, *(float(crs_dict[k]) for k in ('lat_1', 'lat_2', 'lat_0', 'lon_0',)),
                                         *('%0.7f' % f for f in (*ll_tl, *ll_bl, *ll_tr, *ll_br))))
            except rasterio.errors.NotGeoreferencedWarning:
                pass
```

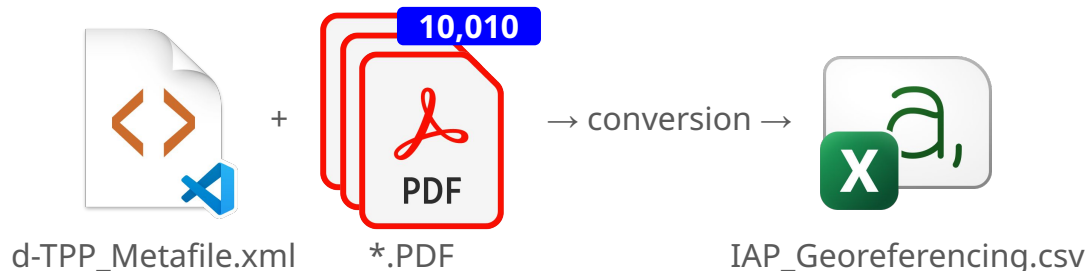
Output Format: CSV, performing conversion



```
% unzip -d tpp_2608 -q DDTPPA_260806.zip
% unzip -d tpp_2608 -q DDTPPB_260806.zip
% unzip -d tpp_2608 -q DDTPPC_260806.zip
% unzip -d tpp_2608 -q DDTPPD_260806.zip
% unzip -d tpp_2608 -q DDTPPE_260806.zip
```

```
% python3 faa_tpp_iap_georef_csv.py tpp_2608 IAP_Georeferencing.csv
[1 minute 38 seconds later...]
```

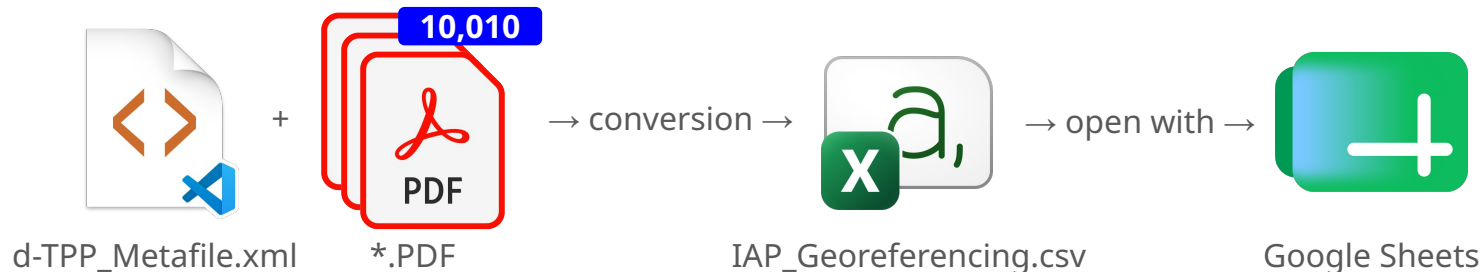
Output Format: CSV, output



Raw CSV

```
filename,sp_lat_1,sp_lat_2,origin_lat,origin_lon,tl_lat,tl_lon,bl_lat,bl_lon,tr_lat,tr_lon,br_lat,br_lon
01244IYLY23.PDF,54.6666666666667,49.3333333333333,51.8835833333333,-176.642472222222,52.3603351,-177.0224124,51.41767
36,-177.0145312,52.3593028,-176.0184025,51.4166626,-176.0313472
01244IZLZ23.PDF,54.6666666666667,49.3333333333333,51.8835833333333,-176.642472222222,52.3603351,-177.0224124,51.41767
36,-177.0145312,52.3593028,-176.0184025,51.4166626,-176.0313472
[...]
00610IL13L.PDF,44.6666667,39.3333333,40.6946389,-73.8686389,41.0696208,-74.2562796,40.1253823,-74.2507098,41.0694826,
-73.4425561,40.1252460,-73.4486782
00610IL22L.PDF,44.6666666666667,39.3333333333333,40.7253055555556,-73.6931666666667,41.0807867,-74.0202725,40.1365428
,-74.0155717,41.0802124,-73.2064118,40.1359769,-73.2134068
00610IL22R.PDF,44.6666666666667,39.3333333333333,40.7263888888889,-73.7098888888889,41.0915435,-74.0377594,40.1472971
,-74.0330468,41.0909742,-73.2237634,40.1467361,-73.2307505
[...]
06048N7.PDF,10.0,9.0,9.49997222222222,138.097583333333,9.9245549,137.7872250,8.9772341,137.7880769,9.9245528,138.4102
771,8.9772320,138.4094189
06048N25.PDF,10.0,9.0,9.49944444444444,138.088472222222,9.9163341,137.7781213,8.9690130,137.7789731,9.9163320,138.401
1585,8.9690109,138.4003003
```

Output Format: CSV, output



Tabular view, as imported into Google Sheets (or any spreadsheet program)

filename	sp_lat_1	sp_lat_2	origin_lat	origin_lon	tl_lat	tl_lon	bl_lat	bl_lon	tr_lat	tr_lon	br_lat	br_lon
01244IYLY23.PDF	54.666667	49.333333	51.883583	-176.642472	52.360335	-177.022412	51.417674	-177.014531	52.359303	-176.018403	51.416663	-176.031347
01244IZLZ23.PDF	54.666667	49.333333	51.883583	-176.642472	52.360335	-177.022412	51.417674	-177.014531	52.359303	-176.018403	51.416663	-176.031347
00610IL4L.PDF	44.666667	39.333333	40.562250	-73.832194	41.014891	-74.201373	40.070665	-74.196073	41.014622	-73.388338	40.070400	-73.394710
00610IL4R.PDF	44.666667	39.333333	40.586778	-73.799528	40.992687	-74.211031	40.048469	-74.205125	40.992724	-73.398272	40.048505	-73.404031
00610IL13L.PDF	44.666667	39.333333	40.694639	-73.868639	41.069621	-74.256280	40.125382	-74.250710	41.069483	-73.442556	40.125246	-73.448678
00610IL22L.PDF	44.666667	39.333333	40.725306	-73.693167	41.080787	-74.020273	40.136543	-74.015572	41.080212	-73.206412	40.135977	-73.213407
00610IL22R.PDF	44.666667	39.333333	40.726389	-73.709889	41.091544	-74.037759	40.147297	-74.033047	41.090974	-73.223763	40.146736	-73.230751
00610IL31L.PDF	45.000000	33.000000	40.590917	-73.684111	40.925960	-74.090196	39.977519	-74.084685	40.925950	-73.274971	39.977509	-73.280523
00610IL31R.PDF	44.666667	39.333333	40.597417	-73.657389	40.993392	-74.062250	40.049174	-74.056439	40.993381	-73.249482	40.049163	-73.255336
06048N7.PDF	10.000000	9.000000	9.499972	138.097583	9.924555	137.787225	8.977234	137.788077	9.924553	138.410277	8.977232	138.409419
06048N25.PDF	10.000000	9.000000	9.499444	138.088472	9.916334	137.778121	8.969013	137.778973	9.916332	138.401159	8.969011	138.400300

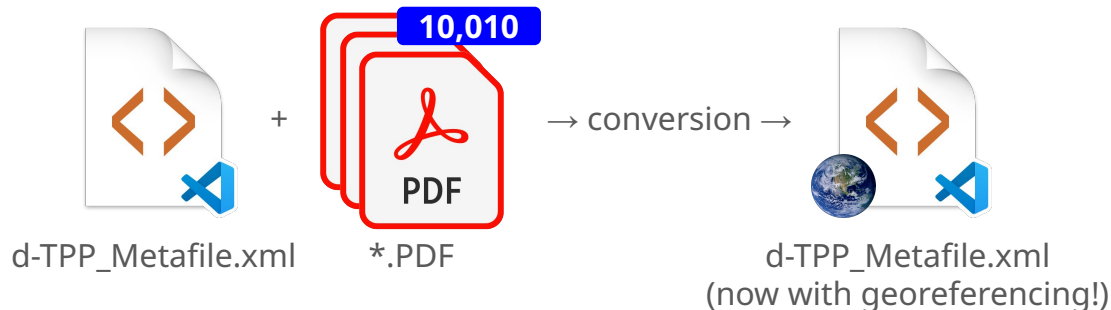
Output Format: d-TPP_Metafile.xml

Benefits

- *Upstream, FAA:* **This file is already being distributed.**
- Users that process FAA IAP charts are **probably already consuming this file**, and are **probably already familiar with it.**
- The “X” in XML is “eXtensible”: it’s possible to **augment the format** without disturbing existing consumers.
- Data can be highly **structured, tailored** to the application.
- It’s possible to **encode the projection and its parameters fully** (for example, as WKT).
- It’s also possible to provide a **decoded form** specific to the Lambert Conformal Conic projection in use.

Drawbacks

- There’s no standard for georeferencing data in XML, so whatever is produced would be “invented”. (*Same as CSV.*)
- Although easy to parse, all users would require custom tooling to harness and use the data. (*Same as CSV.*)
- It’s not as easy as CSV to look at in tabular form.



d-TPP_Metafile.xml (existing format)



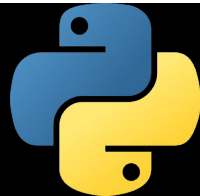
```
<?xml version="1.0" encoding="UTF-8"?>
<digital_tpp cycle="2608" from_edate="0901Z 08/06/26" to_edate="0901Z 09/03/26">
  <state_code ID="NY" state_fullname="NEW YORK">
    <city_name ID="NEW YORK" volume="NE-2">
      <airport_name ID="JOHN F KENNEDY INTL" military="N" apt_ident="JFK" icao_ident="KJFK" alnum="610">
        <record>
          <chartseq>50750</chartseq>
          <chart_code>IAP</chart_code>
          <chart_name>ILS OR LOC RWY 22L</chart_name>
          <useraction></useraction>
          <pdf_name>00610IL22L.PDF</pdf_name>
          <cn_flg>N</cn_flg>
          <cnsection></cnsection>
          <cnpage></cnpage>
          <bvsection></bvsection>
          <bvpage>233</bvpage>
          <procuid>2831</procuid>
          <two_colored>N</two_colored>
          <civil>C</civil>
          <faanfd18></faanfd18>
          <copter></copter>
          <amdtnum>26</amdtnum>
          <amdtdate>12/31/2020</amdtdate>
        </record>
      </airport_name>
    </city_name>
  </state_code>
</digital_tpp>
```

Output Format: d-TPP_Metafile.xml, converter pseudocode

~~Subtractions~~ and additions compared to the CSV converter.

- ~~Write the CSV header, giving column names.~~
- For each <record> in d-TPP_Metafile.xml with <chart_code> = "IAP" and <useraction> ≠ "D":
 - **Open the chart PDF**, using the filename from <pdf_name>.
 - If the PDF is a Geospatial PDF:
 - Optional: Assert that the coordinate reference system uses the Lambert Conformal Conic projection, datum NAD83, and no false easting/northing.
 - Note **projection parameters**: two standard parallels and origin (latitude, longitude).
 - Determine the **projected Cartesian (x, y) coordinates of the page corners** as distances from the projection's origin.
 - Apply the projection's reverse transformation to convert those projected Cartesian (x, y) corner coordinates to **geographic (latitude-longitude) coordinates**.
 - ~~Write an output record for the chart, containing its projection parameters and corner geographic coordinates.~~
 - Add a <georeferencing> element to the <record>, containing the chart's geographic coordinate system definition (WKT), projection parameters, and corner geographic coordinates as control points.
- Output the XML file, including added <georeferencing> elements.

Output Format: d-TPP_Metafile.xml, converter Python code



```
def _xml_el(
    tag: str, attrib: dict[str, str] = {}, text: str | None = None, children: typing.Iterable[xml.etree.ElementTree.Element] = ()
) -> xml.etree.ElementTree.Element:
    el = xml.etree.ElementTree.Element(tag, attrib=attrib)
    el.text = text
    el.extend(children)
    return el

def faa_tpp_iap_georef_xml(tpp_dir: os.PathLike[str] | str, xml_write_file: typing.TextIO) -> None:
    # [...]

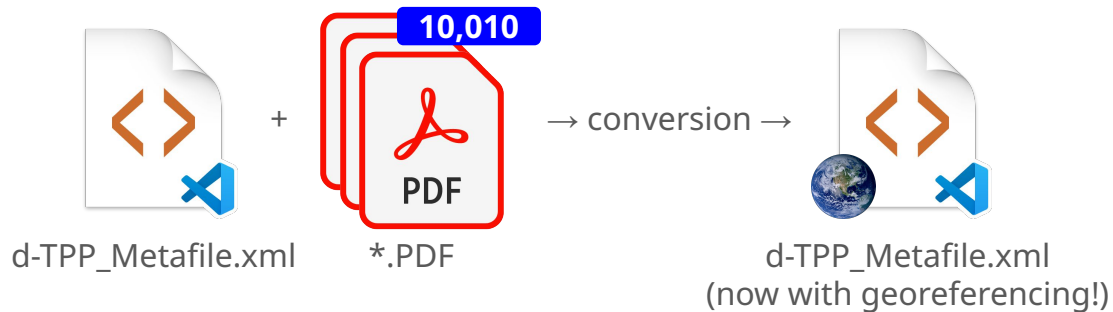
    # (From the CSV example,) for each IAP chart that hasn't been deleted and that is georeferenced:
    ll_tl, ll_bl, ll_tr, ll_br = ((ll_wgs84[1][i], ll_wgs84[0][i]) for i in range(4))

    # Insert this chart's information into the XML structure.
    chart_el.append(
        _xml_el('georeferencing', children=(
            _xml_el('coordinate_reference_system',
                attrib={'format': 'wkt', 'version': 'WKT1_ESRI'}, text=rio.crs.to_wkt(version='WKT1_ESRI')),
            _xml_el('projection', attrib={'type': crs_dict['proj'], 'datum': crs_dict['datum']}, children=(
                _xml_el('standard_parallel', attrib={'latitude': str(float(crs_dict['lat_1']))}),
                _xml_el('standard_parallel', attrib={'latitude': str(float(crs_dict['lat_2']))}),
                _xml_el('origin', attrib={'latitude': str(float(crs_dict['lat_0']))}, 'longitude': str(float(crs_dict['lon_0'])))),
            )),
        _xml_el('control_points', children=(
            _xml_el('control_point',
                attrib={'latitude': '%.7f' % lat, 'longitude': '%.7f' % lon, 'x': str(float(x)), 'y': str(float(y))}
                for (lat, lon), (x, y) in zip((ll_tl, ll_bl, ll_tr, ll_br), ((0, 1), (0, 0), (1, 1), (1, 0)))),
            )),
    )

    except rasterio.errors.NotGeoreferencedWarning:
        pass

    # Write the XML output.
    xml.etree.ElementTree.indent(metafile)
    metafile.write(xml_write_file, encoding='unicode', xml_declaration=True, short_empty_elements=False)
```

Output Format: d-TPP_Metafile.xml, performing conversion



```
% python3 faa_tpp_iap_georef_xml.py tpp_2608 d-TPP_Metafile_Georeferenced.xml  
[1 minute 52 seconds later...]
```

Output Format: d-TPP_Metafile.xml, output, now with georeferencing!

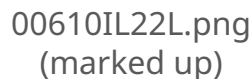


```
<?xml version='1.0' encoding='UTF-8'?>
<digital_tpp cycle="2608" from_edate="0901Z 08/06/26" to_edate="0901Z 09/03/26">
  <state_code ID="NY" state_fullname="NEW YORK">
    <city_name ID="NEW YORK" volume="NE-2">
      <airport_name ID="JOHN F KENNEDY INTL" military="N" apt_ident="JFK" icao_ident="KJFK" alnum="610">
        <record>
          [...]
          <chart_name>ILS OR LOC RWY 22L</chart_name>
          [...]
          <pdf_name>00610IL22L.PDF</pdf_name>
          [...]
          <amtdtdate>12/31/2020</amtdtdate>
          <georeferencing>
            <coordinate_reference_system format="wkt" version="WKT1_ESRI">PROJCS["",GEOGCS["GCS_North_American_1983",DATUM["D_North_American_1983",SPHEROID["GRS_1980",6378137.0,298.2572221]],PRIMEM["Greenwich",0.0],UNIT["Degree",0.0174532925199433]],PROJECTION["Lambert_Conformal_Conic"],PARAMETER["False_Easting",0.0],PARAMETER["False_Northing",0.0],PARAMETER["Central_Meridian",-73.69316666666667],PARAMETER["Standard_Parallel_1",44.66666666666667],PARAMETER["Standard_Parallel_2",39.33333333333333],PARAMETER["Latitude_Of_Origin",40.72530555555556],UNIT["Inch",0.0254000508001]]</coordinate_reference_system>
            <projection type="lcc" datum="NAD83">
              <standard_parallel latitude="44.66666666666667"></standard_parallel>
              <standard_parallel latitude="39.33333333333333"></standard_parallel>
              <origin latitude="40.72530555555556" longitude="-73.69316666666667"></origin>
            </projection>
            <control_points>
              <control_point latitude="41.0807867" longitude="-74.0202725" x="0.0" y="1.0"></control_point>
              <control_point latitude="40.1365428" longitude="-74.0155717" x="0.0" y="0.0"></control_point>
              <control_point latitude="41.0802124" longitude="-73.2064118" x="1.0" y="1.0"></control_point>
              <control_point latitude="40.1359769" longitude="-73.2134068" x="1.0" y="0.0"></control_point>
            </control_points>
          </georeferencing>
        </record>
```

Now that you know how to access the data in the PDFs, it's also possible to use it directly from there without any intermediate steps or files.

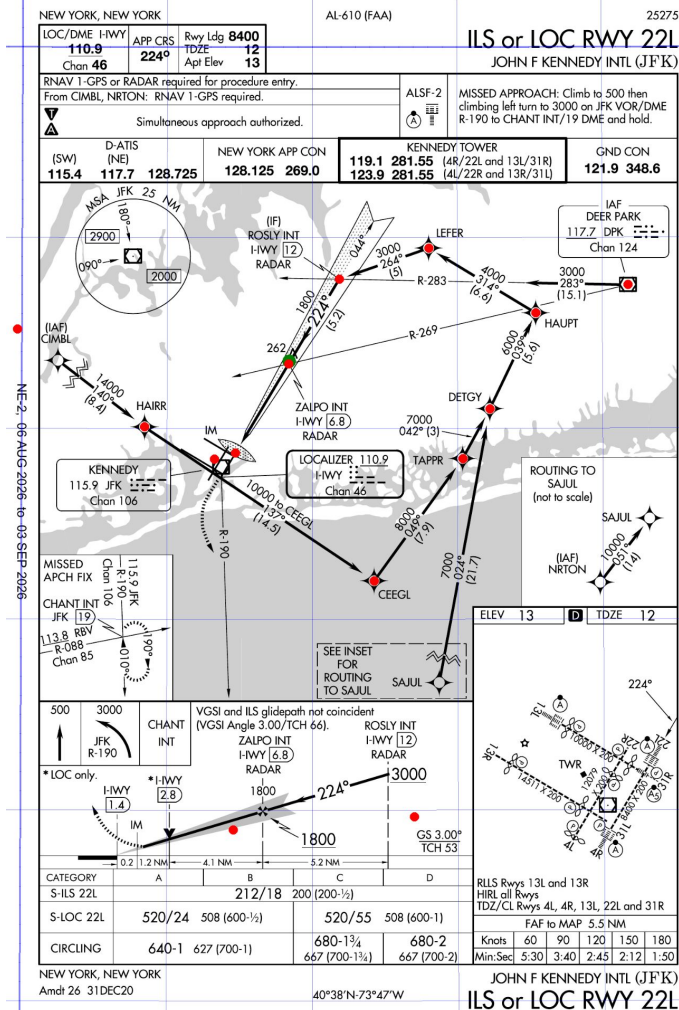
- No additional files to produce or distribute.

- Each new application needs to become aware of Geospatial PDFs.



```
% unzip -d cifp_2608 CIFP_260806.zip
```

```
% python3 faa_tpp_iap_georef_chart.py \
--a424-file=cifp_2608/FAACIFP18 \
--a424-proc=KJFKK6FI22L \
--grid \
00610IL22L.PDF
```



DIY (Do It Yourself): What if you can't or don't want to use GDAL?

- *Are you sure?*
 - **Your GIS software may already use GDAL** under the hood, or make GDAL available.
- *Yes, you're sure.*
 - *Downstream, data suppliers:* Qualifying a large and complex library under [FAA AC 20-153](#) and RTCA DO-200 may be...
 - difficult,
 - time-consuming, or
 - expensive.
 - I'm a fan of open-source software, but you might not be.
 - Uncertain support might be a problem for business-critical processes, particularly if you're not willing or able to build and maintain in-house expertise.
- The GeoTIFF, CSV, d-TTP_Metafile.xml conversion examples and the direct-use example so far have all relied on GDAL.
- Can you **write your own software to extract georeferencing information** directly from Geospatial PDFs?
 - Yes!
 - What do you need in order to do this?
 - A way to **access the structure of a PDF file**. You may already have a library for this, but if not, it's possible to DIY it too.
 - A willingness to do a little math to handle the projection and other transformations. It's not much, and examples are readily available.
 - This isn't theoretical, I've done it.
 - *(And it runs in $\frac{1}{30}$ the time that GDAL does!)*

DIY: Accessing Georeferencing Data



00610IL22L.PDF

%PDF-1.4

8 0 obj

<</Metadata 4 0 R/Names 5 0 R/Outlines 2 0 R/Pages 1 0 R/Type/Catalog>>

endobj

10 0 obj

<</CropBox[0 0 387.36 594]/MediaBox[0 0 387.36 594]/VP[<</BBox[9.18 2.628 378.18 591.372]/Measure<</Bounds[0 0 1 0 1 1 0 1 0 0]/GCS<</EPSG 0/Type/PROJCS/WKT(PROJCS["",GEOGCS["GCS_North_American_1983",DATUM["D_North_American_1983",SPHEROID["GRS_1980",6378137.000,298.25722210]],PRIMEM["Greenwich",0],UNIT["Degree",0.017453292519943295]],PROJECTION["Lambert_Conformal_Conic"],PARAMETER["False_Easting",0.000],PARAMETER["False_Northing",0.000],PARAMETER["Central_Meridian",-73.69316666666671],PARAMETER["Latitude_of_Origin",40.72530555555561],PARAMETER["Standard_Parallel_1",44.66666666666671],PARAMETER["Standard_Parallel_2",39.33333333333331],UNIT["Inch",0.02540005080010]]>>/GPTS[40.23453277014 -73.97048533612 40.23410083011 -73.30825723326 40.98281267452 -73.30381398641 40.98324968235 -73.9231094390 7]/LPTS[0.1 0.1 0.9 0.1 0.9 0.9 0.1 0.9]/PDU[(MI)(SQMI)(DEG)]/Subtype/GeoType/Measure>>>>>>

endobj

1 0 obj

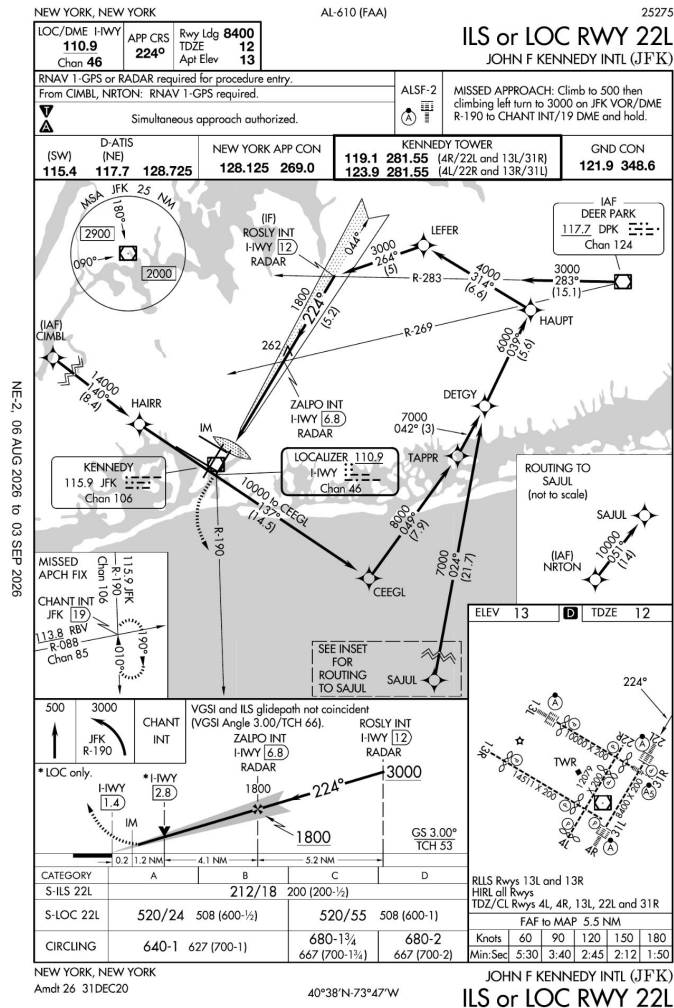
<</Count 1/Kids[10 0 R]/Type/Pages>>

endobj

trailer <</Root 8 0 R>>

%%EOF

start here



%PDF-1.4

```
<</Metadata 4 0 R/Names 5 0 R/Outlines 2 0 R/Pages 1 0 R/Type/Catalog>>
```

```
10 0 obj
```

```
<</CropBox[0 0 387.36 594]/MediaBox[0 0 387.36 594]/VP<</BBox[9.18 2.628  
378.18 591.372]/Measure<</Bounds[0 0 1 0 1 1 0 1 0 0]/GCS<</EPSG 0/Type/  
PROJCS/WKT(PROJCS["",GEOGCS["GCS_North_American_1983",DATUM["D_North_Amer  
ican_1983",SPHEROID["GRS_1980",6378137.000,298.25722210]],PRIMEM["Greenwi  
ch",0],UNIT["Degree",0.017453292519943295]],PROJECTION["Lambert_Conformal  
_Conic"],PARAMETER["False_Easting",0.000],PARAMETER["False_Northing",0.00  
0],PARAMETER["Central_Meridian",-73.693166666666671],PARAMETER["Latitude_O  
f-Origin",40.725305555555561],PARAMETER["Standard_Parallel_1",44.666666666  
66671],PARAMETER["Standard_Parallel_2",39.33333333333331],UNIT["Inch",0.0  
25400005080010]]>>)/GPTS[40.23453277014 -73.92048533612 40.23410083011 -73  
.30825723326 40.98281267452 -73.30381398641 40.98324968235 -73.9231094390  
7]/LPST[0.1 0.1 0.9 0.1 0.9 0.9 0.1 0.9]/PDU[(MI)(SQMI)(DEG)]/Subtype/GEO  
/Type/Measure>>>>>>
```

endobj

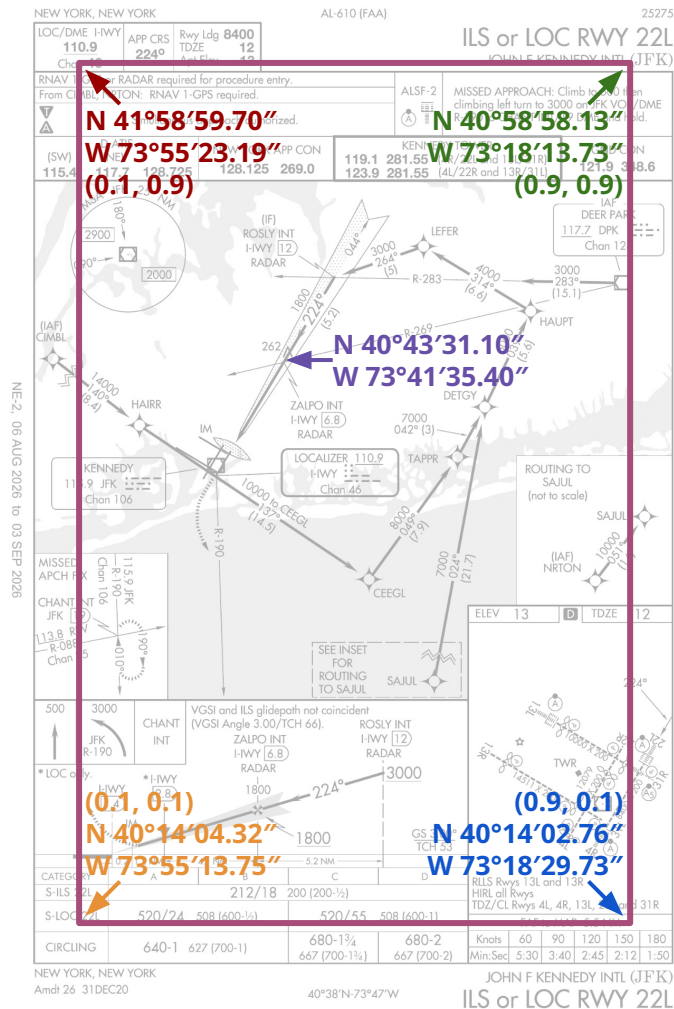
1 0 obj

<</Count 1/Kids[10 0 R]/Type/Pages>>

endobj

```
trailer <</Root 8 0 R>>
```

```
%%EOF
```



DIY: Control Point Conversion



00610IL22L.PDF

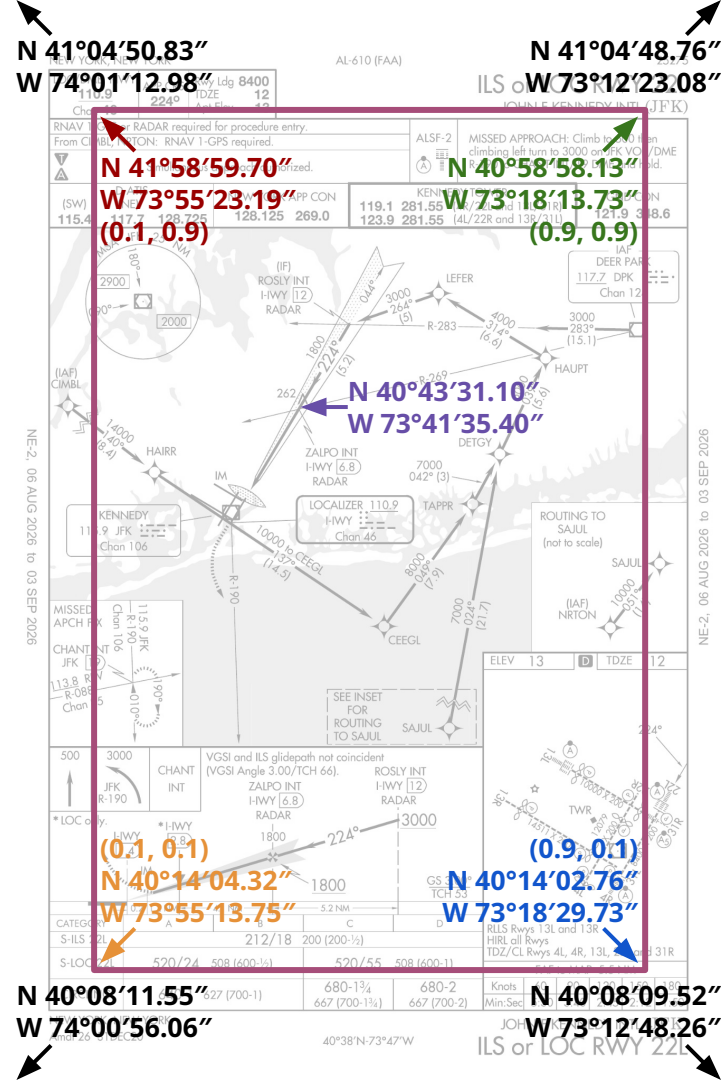
The magenta rectangle is arbitrary and not very useful!
The page's **corner coordinates** would be better, like the previous (GDAL-using) samples provide, and like the recommendation document suggests.

- Set up a Lambert Conformal Conic projection transformation based on the parameters in the PDF (PROJCS).
- Apply the projection's forward transformation to **convert the geographic (latitude-longitude) coordinates** of the magenta box' corners **into projected Cartesian (x, y) coordinates** relative to the **projection origin**.
- **Determine the projected Cartesian (x, y) page corner coordinates** as distances from the **projection origin**.
- Apply the projection's reverse transformation to **convert those projected Cartesian (x, y) corner coordinates to geographic (latitude-longitude) coordinates**.

It's easier than it sounds!

References for Lambert Conformal Conic projection transformations:

- [EPSG Guidance Note 7-2](#), §3.4.1.1, Lambert Conic Conformal (2SP).
- [EPSG Method 9802](#), Lambert Conic Conformal (2SP).
- USGS [Map Projections: A Working Manual](#) (1987), §15, Lambert Conformal Conic.



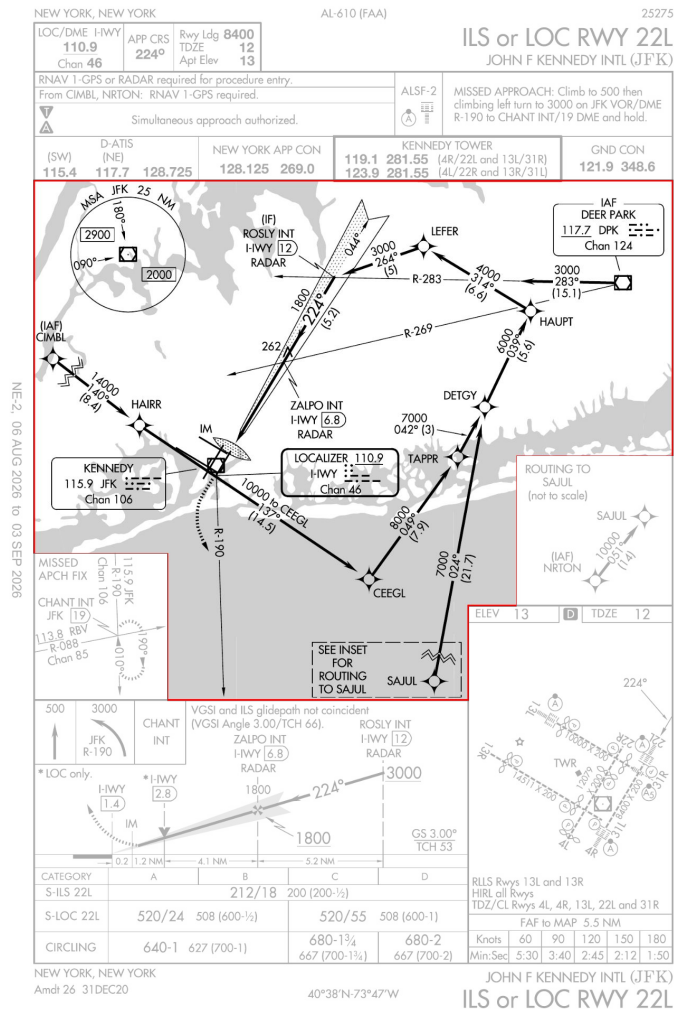
Neatline Extraction



The recommendation document also seeks “ideally, coordinates of the edges of the planview of the chart.”

00610IL22L.PDF

- Geospatial PDF *does* provide a way to store this information (**Bounds**), but it's set to essentially the entire page in FAA instrument approach chart PDFs:
/Bounds[0 0 1 0 1 1 0 1 0 0].
- Technique: **recover the dimensions of the planview** from these PDFs in software, by interpreting the PDFs' drawing commands.
- This can account for and excludes most **insets** (CHANT and NRTON), but not **scale breaks** (CIMBL and SAJUL).
- I've focused this presentation on extracting structured data, so I will not demonstrate this technique in the presentation.
- The image on this slide is intended to illustrate the concept, and was not produced by software.
- *Output format guidance.* Illustrating possibilities for future extensibility:
 - It's possible to **carry this information directly in a Geospatial PDF**.
 - It can also be easily carried in a **further extension to the XML** structure.
 - **CSV is a poor fit for this information**, in particular because the number of points in the shape is variable.



Data Quirks

Oddities in TPP 2608:

- In addition to 10,010 IAPs, there are **4 departure chart pages and 1 arrival chart page** that are georeferenced. **The georeferencing is incorrect.**
 - KPHX BROAK1 (RNAV) departure: [00322BROAK.PDF](#) (procedure 1806, chart 18144).
 - KPHX ECLPS1 (RNAV) departure: [00322ECLPS.PDF](#) (procedure 1806, chart 18144).
 - KPHX FYRBD1 (RNAV) departure: [00322FYRBD.PDF](#) (procedure 1806, chart 18144).
 - KPHX MRBIL1 (RNAV) departure, page 2: [00322MRBIL C.PDF](#) (procedure 1806, chart 18144, no plan view!).
 - KONT SCBBY2 (RNAV) arrival, page 2: [00965SCBBY C.PDF](#) (procedure 1802, chart 18032).
- All georeferenced charts are “true north up” (not rotated) except for one. GDAL handles this well, but it would take more effort for handmade GIS tools to accommodate this.
 - NSTU/PPG approach ILS or LOC 5: [05018IL5.PDF](#) (procedure 2108, chart 26134), the plan view is **rotated 0.16° clockwise**.

Source Code Availability

The sample code included in this presentation, along with its output, is available under the Apache 2.0 license at https://github.com/markmentovai/faa_tpp_iap_georef/.